

資料庫期末計劃書：電商訂單與庫存管理系統

組員：陳昇宏、夏承冠、賴威凱、蘇綵蓁

一、專案主題與問題定義

1. 專案主題、應用情境與想解決的資料管理問題

- 專案主題：電商訂單與庫存管理系統
- 應用情境：本系統主要應用於「中小型電商賣家」或「個人網拍工作室」的日常營運後台。當商家在網路上販售多種商品，且每天面臨來自四面八方的顧客下單時，需要一個能即時處理進銷存與訂單狀態的資訊環境。
- 想解決的資料管理問題：傳統或無系統輔助的小型商家常依賴人工（如試算表或紙本）紀錄，我們希望能透過關聯式資料庫解決以下三大痛點：
 1. 商品庫存不易掌握：商品品項繁多時，無法即時得知各項商品的確切剩餘數量。
 2. 訂單狀態混亂：顧客從下單、付款到出貨的流程缺乏統一的追蹤介面，容易造成處理進度混淆。
 3. 人工更新庫存容易錯誤：依賴人工在訂單成立後手動扣減庫存，極易發生計算錯誤，甚至導致「超賣」或漏單的嚴重失誤。

2. 選題動機與對實務/使用者的價值

- 選題動機：觀察目前的市場現況，大型電商平台（如 Shopee、PChome）雖然功能十分齊全，但對於部分小型商家來說，其後台系統可能過於龐雜且缺乏彈性；另一方面，市面上簡易的購物車系統又往往只專注於結帳，缺少完整且嚴謹的庫存紀錄與訂單連動機制。因此，我們希望打造一款介於兩者之間，專注於解決「訂單與庫存核心關聯」的輕量化管理系統。
- 對實務或使用者的價值：
 1. 防範營運風險（資料一致性）：透過資料庫系統在訂單成立時自動精準扣除庫存，有效避免人為疏失造成的「超賣」客訴問題。
 2. 提升管理效率：將訂單明細、付款狀態與庫存管理整合在同一個系統下，讓商家能快速查詢與更新狀態，大幅節省人工對帳的時間。
 3. 數據化營運基礎：結構化的資料庫設計（如會員、商品分類、訂單明細）能為商家保留完整的歷史交易紀錄，未來可作為分析熱銷商品或制定進貨策略的重要依據。

二、需求分析與資料庫設計規劃

1. 系統主要功能與資料庫類型

主要功能模組：

- 商品管理：提供商品的建檔、上下架狀態切換，以及商品分類的維護。
- 會員下單：支援會員註冊、登入與購物車結帳流程，確保訂單能準確綁定購買人。
- 訂單明細與付款狀態查詢：紀錄每一筆訂單的購買品項、單價、數量與總金額，並即時追蹤「未付款、已付款、處理中、已完成」等狀態。
- 庫存扣減與紀錄管理：系統需自動化處理訂單成立後的庫存扣減，並留下不可篡改的庫存變動紀錄(如進貨、售出、退貨)，以解決人工更新易出錯的問題。

資料庫類型選擇：考量到電商系統對於交易一致性(ACID特性)有極高的要求，尤其是處理訂單扣款與庫存扣減時不能有任何差錯，建議採用關聯式資料庫。這能讓我們透過嚴謹的資料表結構與正規化設計(Normalization)，來避免資料異常與重複儲存，同時確保多位使用者同時下單的高併發環境下，資料依然保持完整與正確。

2. 系統設計原則

正規化守則 (Normalization):

- 第一正規化 (1NF) - 確保欄位「不可分割」
 - 守則：每一個欄位都只能儲存單一值(不可再分的原子值)，且同一張表不能有重複性質的欄位組。
 - 第二正規化 (2NF) - 消除「部分相依」
 - 守則：必須先符合 1NF。每一張表必須有主鍵(Primary Key, PK)，且所有非主鍵的欄位，都必須「完全依賴」整個主鍵，不能只依賴主鍵的一部分(這通常發生在複合主鍵的情況)。
 - 第三正規化 (3NF) - 消除「遞移相依」
 - 守則：必須先符合 2NF。非主鍵欄位之間不能有互相依賴的關係，也就是非主鍵欄位只能依賴主鍵。
-

資料完整性守則 (Data Integrity):

=> 確保資料庫不會出現「幽靈資料」或「孤兒資料」的鐵則

實體完整性 (Entity Integrity):

- 每張資料表都必須有一個主鍵 (Primary Key)，用來唯一識別每一筆記錄。
- 主鍵絕對不能是空值(Null)，且不能重複。例如：每一筆訂單都要有獨一無二的 **訂單ID**。

參照完整性 (Referential Integrity):

- 善用外鍵 (**Foreign Key**) 來約束資料關聯。
- 例如:「訂單表」裡的 會員ID, 必須真實存在於「會員表」中。資料庫會幫你擋下「試圖把訂單綁定給一個不存在的會員」這種錯誤操作。

網域完整性 (Domain Integrity):

- 設定正確的資料型態與限制條件。
- 例如:「商品庫存量」必須設定為整數 (INT), 且可以加上 ≥ 0 的限制 (Check Constraint), 防止庫存被扣成負數。「電子郵件」欄位可以設定字串長度與唯一性 (Unique)。

關聯設計與實務守則:

1. 破解「多對多 (N:M)」的魔咒:
 - 關聯式資料庫無法直接處理多對多的關係, 必須透過建立「中介表」來拆解成兩個一對多 (1:N) 的關係。
2. 命名規範 (Naming Conventions):
 - 保持一致性: 表名和欄位名稱盡量統一使用英文小寫, 單字之間用底線分隔 (Snake Case, 例如 `order_details`、`member_id`)。
 - 見名知義: 避免使用 `table1`、`data2` 這種毫無意義的命名。
3. 【進階實務】適度的「反正規化」 (Denormalization):
 - 雖然 3NF 是基本功, 但在實務上, 有時候為了「查詢效能」或「保留歷史快照」, 我們會故意打破正規化。
 - 例子: 按照 3NF, 「訂單明細表」不該記錄「單價」, 因為單價在「商品表」已經有了。但是不把當下結帳的單價寫入「訂單明細」, 未來商品漲價時, 歷史訂單總額會跟著變動, 在會計上是災難。

3. 初步實體、屬性、關係與資料表設計

A. 實體與屬性規劃 (Entities & Attributes)

1. 會員 (Members)
 - 屬性: 會員ID (PK)、姓名、電子郵件、密碼雜湊值、聯絡電話、送貨地址、註冊時間。
2. 商品分類 (Categories)
 - 屬性: 分類ID (PK)、分類名稱、分類描述。
3. 商品 (Products)

- 屬性: 商品ID (PK)、分類ID (FK)、商品名稱、售價、當前庫存量、商品描述、上架狀態(是/否)。
- 4. 訂單 (Orders)
 - 屬性: 訂單ID (PK)、會員ID (FK)、訂單總金額、付款狀態(未付/已付/失敗)、訂單狀態(處理中/已出貨/已完成)、建立時間。
- 5. 訂單明細 (Order_Details)
 - 屬性: 明細ID (PK)、訂單ID (FK)、商品ID (FK)、購買數量、結帳單價、小計金額。
 - 設計考量: 紀錄結帳當下的「單價」, 避免未來商品調價時影響歷史訂單的金額計算。
- 6. 庫存紀錄 (Inventory_Logs)
 - 屬性: 紀錄ID (PK)、商品ID (FK)、變動數量(正值或負值)、變動類型(進貨/訂單扣減/取消退回)、紀錄時間。
 - 設計考量: 專門解決「庫存不易掌握」的痛點, 每一筆庫存增減都有跡可循。

B. 實體關聯設計 (Relationships)

- 會員與訂單 =>【下單】關係 (1:N)

描述: 會員「下單」產生訂單。一個會員可以在系統中下單多次(產生多筆訂單), 但每一筆已成立的訂單, 必定明確隸屬於某一位特定的會員。

- 商品分類與商品 =>【包含 / 歸屬】關係 (1:N)

描述: 分類「包含」商品。為了方便使用者搜尋與管理, 一個主分類下會包含多種不同商品, 而每一項商品在建檔時, 都必須「歸屬」於一個特定的商品分類中。

- 訂單與訂單明細 =>【記載】關係 (1:N)

描述: 訂單「記載」了訂單明細。一筆完整的訂單通常包含消費者一次購買的多個品項, 這些品項會被拆解並「記載」成多筆訂單明細。

- 商品與訂單明細 =>【列入】關係 (1:N)

描述: 商品被「列入」訂單明細中。同一項熱門商品可能會被不同的消費者購買, 因此會「列入」在多筆不同的訂單明細紀錄中。(註: 透過此關係, 完美解決了訂單與商品之間的多對多 N:M 問題)

- 商品與庫存紀錄 =>【產生】關係 (1:N)

描述: 商品異動「產生」庫存紀錄。無論是進貨、售出扣減還是退貨, 只要該商品的庫存數量發生變動, 就會「產生」一筆對應的歷史庫存紀錄。

C. 參與 (Participation) 與基數 (Cardinality) 分析

以 (最小參與度, 最大基數) 的格式來標示。以下為本系統各關聯的詳細定義：

會員「下單」訂單

- 會員端的限制為 (0, N)：基數為 N，參與為選擇性 (Optional)。一個會員註冊後可以還沒買過東西 (0 筆訂單)，也可以買很多次 (N 筆訂單)。
- 訂單端的限制為 (1, 1)：基數為 1，參與為強制 (Mandatory)。只要系統中存在一筆訂單，必定且只能對應到 1 位確切的會員。

分類「包含」商品

- 分類端的限制為 (0, N)：基數為 N，參與為選擇性 (Optional)。建立一個新的分類後，裡面可能暫時還沒有商品上架 (0 個)，也可能包含極多商品 (N 個)。
- 商品端的限制為 (1, 1)：基數為 1，參與為強制 (Mandatory)。每一個建檔的商品，都強制要求必須指派給 1 個主分類，不能沒有分類。

訂單「記載」訂單明細

- 訂單端的限制為 (1, N)：基數為 N，參與為強制 (Mandatory)。一筆訂單一旦成立，裡面至少要包含 1 筆購買品項 (明細)，最多則可以有 N 筆不同品項。
- 明細端的限制為 (1, 1)：基數為 1，參與為強制 (Mandatory)。每一筆訂單明細紀錄，必定只能隸屬於 1 筆特定的母訂單。

商品被「列入」訂單明細

- 商品端的限制為 (0, N)：基數為 N，參與為選擇性 (Optional)。一項商品上架後，可能還沒被任何人買過 (出現在 0 筆明細中)，也可能非常熱銷 (出現在 N 筆明細中)。
- 明細端的限制為 (1, 1)：基數為 1，參與為強制 (Mandatory)。訂單明細中的每一行紀錄，都必定只能對應到 1 種具體的商品實體。

商品「產生」庫存紀錄

- 商品端的限制為 (0, N)：基數為 N，參與為選擇性 (Optional)。(假設新商品剛建檔且尚未進貨時，可能還沒有庫存變動軌跡；但只要有異動就會有 N 筆紀錄)。
- 紀錄端的限制為 (1, 1)：基數為 1，參與為強制 (Mandatory)。每一筆寫入系統的庫存變動紀錄，都必須明確指出是哪 1 個商品的庫存發生了變化。

三、現有系統或相關作品分析

目前常見的電商訂單與庫存管理系統包含 Shopee、PChome 等大型電商平台，以及一般簡易購物車系統。

Shopee 提供完整的電商功能，包含商品上架、訂單管理、付款與物流整合等，系統功能齊全且適用於大型商業營運。然而其系統架構複雜，對於教學或小型專案而言較難理解與實作，且後台資料結構不易取得，不利於資料庫設計之學習。

PChome 則以快速購物與訂單處理為主要特色，具備完善的商品分類與搜尋功能，但其設計偏重於使用者購物體驗，對於後台資料管理流程與庫存控管機制的呈現較為有限。

此外，一般簡易購物車系統多僅提供商品瀏覽與下單功能，雖然架構較為單純，適合初學者開發，但常缺乏完整的庫存管理機制與訂單狀態控管，例如未明確區分付款、出貨與完成等狀態，亦缺少歷史訂單查詢與分析功能。

綜合上述分析，現有系統雖各具優點，但仍存在系統過於複雜或功能不足等問題。因此，本專題將設計一套結構清晰且功能完整的電商訂單與庫存管理系統，著重於資料庫設計與資料關聯性，並強化庫存控管與訂單狀態管理機制，以提升系統實用性與學習價值。

本組專題之差異與改進方向

綜合前述現有系統分析，本組專題在設計上將針對現有電商系統之不足進行改進，主要差異與方向如下：

一、系統架構簡化與教學導向設計

現有大型電商平台功能完整但系統架構複雜，不利於理解與實作。本專題將以教學與實作為導向，設計結構清晰之資料庫系統，使各資料表與其關聯關係明確，提升系統可讀性與學習價值。

二、強化庫存管理機制

部分簡易購物系統缺乏完整庫存控管功能，容易造成商品數量錯誤或超賣問題。本專題將設計庫存即時更新機制，於訂單成立時自動扣除庫存，並可查詢庫存變動紀錄，以提升資料正確性。

三、完善訂單狀態管理

現有系統中，部分簡易系統未明確區分訂單處理流程。本專題將建立完整訂單狀態，例如「未付款」、「已付款」、「已出貨」與「已完成」，以利使用者與管理者追蹤訂單進度。

四、提升資料查詢與管理效率

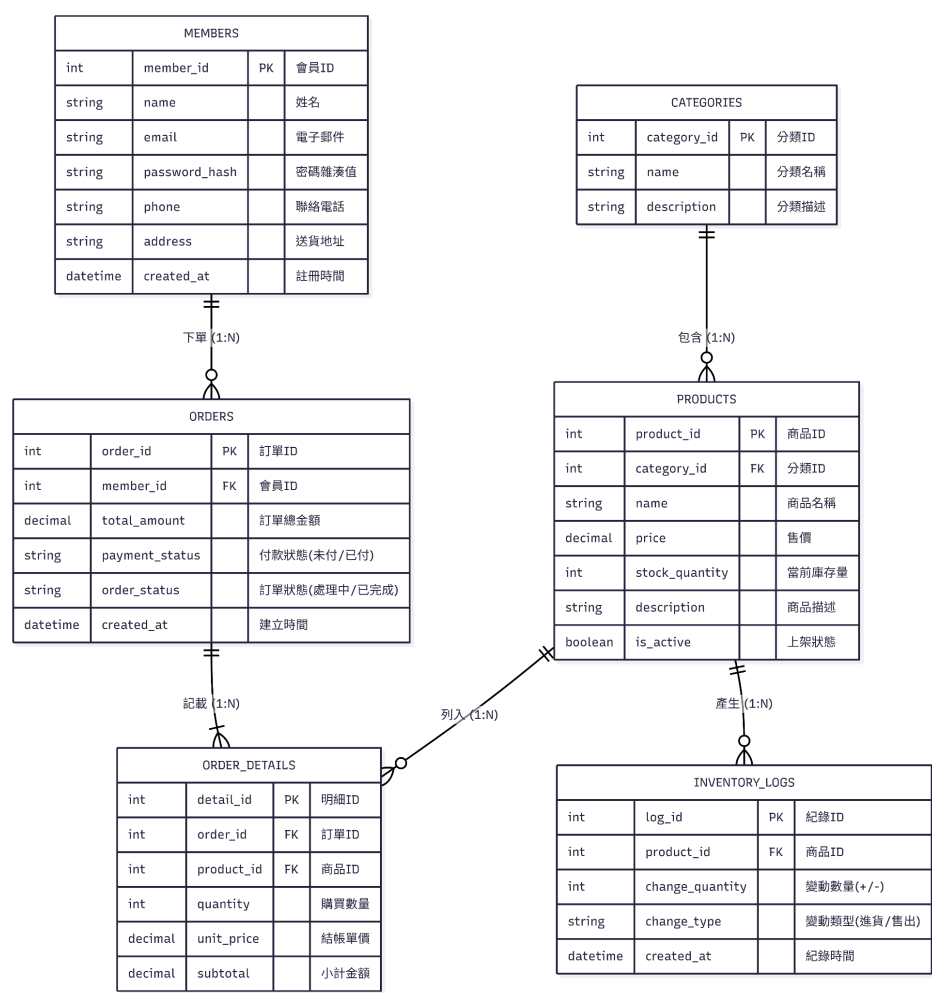
本專題將提供基本查詢功能，如訂單查詢、商品查詢與庫存查詢，使管理者能快速掌握系統狀態，並提升整體管理效率。

綜上所述，本專題透過簡化系統架構並強化核心功能，期望建構一套兼具實用性與教學價值之電商訂單與庫存管理系統。

四、預期成果與執行挑戰

1. 預期完成之成果 本組專題預計將產出以下具體成果，以驗證系統之可行性與資料庫設計之完整性，具體包含 ERD、資料表與核心查詢功能：

- 完整的實體關聯圖 (ERD)：依據先前的需求分析，繪製包含會員、商品分類、商品、訂單、訂單明細與庫存紀錄等 6 個實體的 ERD。圖中將清楚標示主鍵 (PK)、外鍵 (FK)、各項屬性，以及實體間的基數 (Cardinality) 與參與度 (Participation) 關係。



- 實體資料表建置 (Data Tables)：於關聯式資料庫系統(如 MySQL、SQL Server 等)中實際建立正規化後的資料表。我們將嚴格落實資料完整性約束，包含設定正確的資料型態、Unique 限制、Check 限制(如庫存數量 ≥ 0)，並正確設定 Foreign Key 以確保參照完整性。

- 核心查詢功能與 SQL 實作：針對電商營運情境，撰寫並測試對應的 SQL 語法，以模擬實際的系統介面或查詢功能。預計完成的關鍵查詢包含：
 - 訂單明細還原：透過 JOIN 語法關聯會員、訂單、訂單明細與商品表，完整呈現單筆訂單的購買品項、結帳單價與總額。
 - 即時庫存與異動追蹤：查詢特定商品的當前可用庫存，並能調閱 Inventory_Logs 表追蹤該商品的所有進、銷、退紀錄。
 - 營運統計分析：使用 GROUP BY 與聚合函數 (SUM, COUNT) 計算各分類商品的銷售總額，或列出特定區間內的熱銷商品排行。

2. 可能遇到的困難與執行挑戰 在專案執行與實作過程中，我們預期在需求分析、資料表設計或查詢效能方面可能面臨以下挑戰：

- 資料表設計與商業邏輯的兩難：為了確保歷史訂單金額不受未來商品漲價影響，我們已在「訂單明細」中設計了「結帳單價」欄位（適度的反正規化）。然而，若未來需擴充「滿額折扣」或「優惠券抵扣」等複雜情境時，如何設計適當的欄位來記錄這些變動金額，且不過度破壞正規化原則，將是資料表設計上的進階挑戰。
- 庫存一致性與交易併發處理：電商系統最容易發生「超賣」的致命錯誤。在多人同時下單搶購同一件商品的極端情境下，如何確保資料庫在讀取與更新庫存時不會發生競賽條件 (Race Condition)。我們可能需要進一步了解資料庫的交易 (Transaction) 控制與鎖定 (Lock) 機制，以確保庫存扣減的絕對正確。
- 複雜查詢的效能瓶頸：當系統營運一段時間、訂單與庫存紀錄的資料量大幅增加後，若要頻繁進行多張資料表的 JOIN 查詢，或統計歷史銷售額，可能會遭遇查詢效能低下的問題。未來在實作時，我們可能需要評估並學習如何針對常作為查詢條件的欄位（如訂單狀態、商品分類、日期）建立索引 (Index)，以優化檢索速度。